Lab 7.2: Using CDynamicAccessor

In this lab, you will create a consumer application by using the **CDynamicAccessor** class. You will retrieve data from a data source whose structure is not known at design time.

To see a demonstration of the solution to this lab, click this icon.
(CD-ROM plays the demonstration, "Lab 7.2 Demonstration.")

Estimated time to complete this lab: **30 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch07\Lab7.2\Ex01.

The solution code for this lab is located in the folder \Labs\Ch07\Lab7.2\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Create a consumer application by using the **CDynamicAccessor** class.
- Retrieve data from a data source.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter.

Exercise 1: Retrieving Data from a Data Source

In this exercise, you will retrieve data from a data source whose structure is not known at design time.

236 Exercise 1: Retrieving Data from a Data Source

The code that forms the basis for this exercise is in \Labs\Ch07\Lab7.2\Ex01.

In this exercise, you will retrieve data from a data source whose structure is not known at design time.

▶ **Use CDynamicAccessor to retrieve data from a data source**

☐ 1. Open the fileOLEDBDynAccAppView.h.

☐ 2. Declare a protected member function to retrieve data from a data source:

```
void RetrieveRecords();
```

☐ 3. Open the file OLEDBDynAccAppView.cpp.

☐ 4. In the **RetrieveRecords** function, create an object of type **CDBPropSet** as follows:

```
CDBPropSet  propset(DBPROPSET_ROWSET);
```

☐ 5. Declare two **char** arrays to store the field values as follows:

```
char disp_str[500], TempStr[500];
```

☐ 6. Declare a pointer to the Field structure as follows:

```
struct Field *pFieldData;
```

The Field structure is defined in OLEDBDynAccAppView.h as follows:

```
struct Field
{
    void *pField;
    DBTYPE ColType;
};
```

*pField is a pointer to the location in the buffer where the field value is stored. ColType stores the field data type.

☐ 7. Obtain a pointer to the list box that will be used to display data from the database table as follows:

```
m_plbData=(CListBox*)GetDlgItem(IDC_DATA);
m_plbData->ResetContent();
```

☐ 8. Use the **AddProperty** member function of **CDBPropSet** object to set properties for the accessor as follows:

```
propset.AddProperty(DBPROP_CANFETCHBACKWARDS, true);
```

DBPROP_CANFETCHBACKWARDS allows backward fetching and scrolling in nonsequential rowsets.

☐ 9. Declare a variable of **CCommand<CDynamicAccessor>** class as follows:

```
CCommand<CDynamicAccessor> da;
```

☐ 10. Invoke the **CCommand<CDynamicAccessor>** member function **Open** to access the records in the data source table as follows:

```
if (S_OK != da.Open(m_session,m_cmd_str,&propset))
{
    MessageBox("Failed to Open");
    return;
}
```

Notes

☐ 11. Retrieve the number of columns in the database table as follows:

```
int nColCount=da.GetColumnCount();
```

☐ 12. Create instances of the Field structure based on the number of columns as follows:

```
pFieldData = new Field[nColCount];
```

☐ 13. Obtain the column type of each field in the table as follows:

```
for(int i = 1; i < nColCount; i++)
{
    da.GetColumnType(i, &((pFieldData + (i - 1))->ColType));
}
```

☐ 14. Retrieve the value of each field and add the value to the list box after typecasting it to appropriate data type.

To see an example of how the completed **RetrieveRecords** function should look, click this icon. (CD-ROM displays the following sample code.)

```
void COLEDBDynAccAppView::RetrieveRecords()
{
    CDBPropSet  propset(DBPROPSET_ROWSET);
    char disp_str[500], tempstr[500];
    struct Field *pFieldData;

    m_plbData=(CListBox*)GetDlgItem(IDC_DATA);
    m_plbData->ResetContent();

    propset.AddProperty(DBPROP_CANFETCHBACKWARDS, true);
    propset.AddProperty(DBPROP_IRowsetScroll, true);


    CCommand<CDynamicAccessor> da;

    if (S_OK != da.Open(m_session,m_cmd_str,&propset))
    {
        MessageBox("Failed to Open");
        return;
    }

    int nColCount=da.GetColumnCount();
    pFieldData = new Field[nColCount];


    da.MoveFirst();
    for(int i = 1; i < nColCount; i++)
    {
        da.GetColumnType(i, &((pFieldData + (i - 1))->ColType));
    }
```

(Code continues on following page.)

Notes

```
    tempstr[0]='\0';

    do
    {
        disp_str[0]='\0';
        for(int i =1 ; i < nColCount; i++)
        {
            switch ((pFieldData + (i - 1))->ColType)
            {
            case DBTYPE_I2: (pFieldData + (i - 1))->pField = da.GetValue(i);
                            sprintf(tempstr," %d ",*((int*)((pFieldData + (i
- 1))->pField)));
                            break;
            case DBTYPE_I4: (pFieldData + (i - 1))->pField = da.GetValue(i);
                            sprintf(tempstr," %d ", *((long*)((pFieldData +
(i - 1))->pField)));
                            break;
            default:    (pFieldData + (i - 1))->pField = da.GetValue(i);
                            sprintf(tempstr," %s ", (pFieldData + (i - 1))-
>pField);
                            break;
            }
            strcat(disp_str, tempstr);
        }

        m_plbData->AddString(disp_str);
    }while(da.MoveNext()==S_OK);
}
```

Notes

☐ 15. Invoke **RetrieveRecords** function from **OnFileOpen**.

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void COLEDBDynAccAppView::OnFileOpen()
{

    if(enumDlg.DoModal()==IDOK)
    {
        datasourceDlg.m_RootEnum = &(enumDlg.RootEnum);
        if(datasourceDlg.DoModal()==IDOK)
        {
            tablesDlg.m_session = &(datasourceDlg.m_session);
            tablesDlg.m_source = &(datasourceDlg.m_source);
            if(tablesDlg.DoModal()==IDOK)
            {
                columnsDlg.m_session = tablesDlg.m_session;
                m_session = *(tablesDlg.m_session);
                columnsDlg.m_Tbl = &(tablesDlg.Tbl);
                columnsDlg.DoModal();
                m_cmd_str =  columnsDlg.m_cmd_str;
                RetrieveRecords();
            }

        }

    }
}
```

▶ **Build and test your application**

- Run your application. On the **File** menu, click **Open** to retrieve data from a data source.

The solution code for this exercise is located in the folder \Labs\Ch07\Lab7.2\Ex01\Solution.

Notes